Evo Hash: A Blockchain Whose Chain Is the Model

Cole Heideman
vibehash.org

Abstract. Proof-of-work spends its computation on hashes that are thrown away, and attempts to spend it on training neural networks have been held back by gradients: they are gigabytes per step, too large to send over home connections and too opaque to check. We propose training by evolution strategies instead, where a step of training is fully described by a handful of seeds and the scores of the perturbed models those seeds define. A block then holds one generation of evolution in about two kilobytes, every node applies the same update from it, and the trained model is a pure function of the chain: anyone can rebuild every weight exactly, so no one stores, ships or owns it. Miners answer problems with perturbed copies of the model, checkers replay a random sample of those answers, and grading is a few lines of code any node can run. On one consumer graphics card, a test network trains all 357,826,560 projection weights of a 0.5-billion-parameter language model at about eighteen seconds a generation, and records 43 generations in 95.8 KB.

1. Introduction

Bitcoin [1] keeps a ledger without a trusted party by making blocks expensive to produce, and the expense buys nothing else. Vibe Hash [2] replaced the hashes with the training of neural networks, verified by replaying committed checkpoints. That design pays for useful work, but every unit of it still moves model-sized checkpoints and gradients across the network, and checking a unit means repeating its arithmetic within a tolerance.

Evolution strategies (ES) [3] train a model without gradients. Each step evaluates many copies of the model, each moved a small random distance from the current weights, and moves the weights toward the copies that scored well. ES was long thought too slow for large language models; recent work has shown it fine-tunes billion-parameter models competitively [4, 5]. Its property that matters here is older: each random move is generated from a seed, so a step is determined by its seeds and its scores alone, a fact zeroth-order fine-tuning [10] and seed-based federated tuning [11] have used to cut memory and communication. In this paper we make that property the basis of a chain. A block records a generation's seeds and scores, the update is a deterministic function of the block, and the model at any height is the base model plus every update before it. Mining becomes the search, and the chain becomes the model.

2. Jobs

A requester posts a job naming a pinned base model, the tensors to train, a task, and the evolution settings: the number of antithetic pairs P, problems per round Q, the noise scale σ, a step size that

falls along a cosine from η to a quarter of η over the job's planned rounds, and a fee per round held in escrow. A job may also name parents, earlier jobs whose final change it starts from, each pinned by the hash of its final state and given a weight. The defaults are $P = 64$ pairs (128 members), $Q = 16$ problems of at most 64 generated tokens, $\sigma = 10^{-3}$ and $\eta = 4 \times 10^{-3}$, training every attention and MLP projection of the model, 168 matrices for Qwen2.5-0.5B-Instruct.

A task supplies problems drawn deterministically from a seed and a check of each reply that is pure code: an exact expression for Countdown, a number for arithmetic and word problems, a letter for multiple choice, and for trivia a match, after normalisation [6], against the dataset's accepted answers. Datasets are pinned to one commit, so every machine draws the same questions, and training questions come only from splits that scoring never uses.

3. Members

For a pair j and each trained matrix W of shape $m \times n$, the pair's seed generates two vectors $a_j \in \mathbb{R}^m$ and $b_j \in \mathbb{R}^n$, and the pair's two members use

$$W_j^{\pm} = W \pm \sigma\, a_j\, b_j^T,$$

a rank-one twist of every matrix at once, after EGGROLL [5]. The twist never has to be formed: a member's product $W_j^{\pm}x$ is Wx plus $\sigma(b_j^T x)a_j$, so all 128 members run as one batch through the shared base weights with a small correction each. A pair's vectors for every matrix come from one stream of about 550,000 numbers.

All members read each problem's prompt, except its last token, through the untwisted model, once; the twists apply from the last prompt token on. This shared-prompt rule is part of a member's definition, not an approximation a worker may choose, and it makes 128 members cost little more than one. Pair seeds are derived from the round seed and the pair's index, and the round seed is

$$seed = H(\text{"es-round"} \,\|\, previous\ block \,\|\, round\ number \,\|\, registrations\ ),$$

fixed when registration closes and unknown before it, so no worker can choose its noise.

4. A Round

Each round produces one block, and each block is one generation (Figure 1). The round's pairs are cut into units of eight pairs, assigned to registered workers as in Vibe Hash. A worker runs its unit's sixteen members on the round's problems, uploads their answers as a content-addressed blob, and commits to their scores before the reveal. During verification, checkers drawn at random replay a sample of each unit's members: they feed each answer back through that member's twisted model and require every generated token to lie within a margin of the member's greedy choice, which an answer the member would not have produced fails. The node regrades every answer with the task's check, so a worker cannot claim a right answer it did not give.

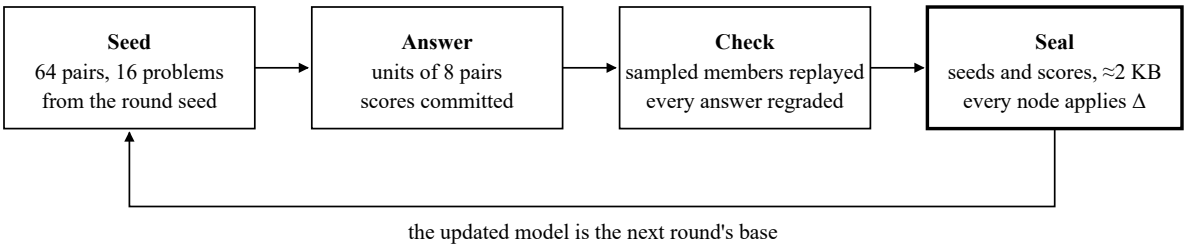


Figure 1: One round. Answering and checking run on workers' graphics cards; the update is a deterministic function of what the block records, so every node applies it identically.

From the accepted pairs the node computes the update. Each member's fitness is shaped by z-scoring across the generation, and pair j's coefficient is the difference between its members' shaped scores, $c_j = z_j^{+} - z_j^{-}$. Every matrix then moves by

$$\Delta W = (\eta_t / N) \, \Sigma_j \, c_j \, a_j \, b_j^{T},$$

where η_t is the step size at round t and N the number of accepted members. Taking differences within a pair cancels most of the noise that the choice of problems puts into the scores.

A right-or-wrong score is coarse: when a model answers 5% of problems correctly, most pairs score alike and the generation learns little. Each round therefore also keeps, for every problem, the most common correct answer its members gave. In later rounds a member's fitness is its mean log-probability of the previous rounds' kept answers, a graded signal that rewards becoming more confident in answers already verified correct. Checkers recompute these log-probabilities for their sampled members within a tolerance.

5. The Chain Is the Model

A round's entry in its block is its seed, the list of accepted pairs with their coefficients, the step size and the replay targets. Nodes never hold weights. A job's on-chain state is a hash chain of 32 bytes, stored as sixteen 16-bit words, followed by eight control words that count its steps:

$$h_0 = H(\text{"vibehash-es2-start"} \,\|\, base \,\|\, tensors \,\|\, parents\,), \quad h_{t+1} = H(\, h_t \,\|\, entry_t\,).$$

Each block commits to the job's state after its round, so a node that misreads any entry disagrees with the block at once. A worker that needs the weights rebuilds them: starting from the base model and its parents' changes, it regenerates each round's vectors from the recorded seeds and applies each recorded update in order (Figure 2).

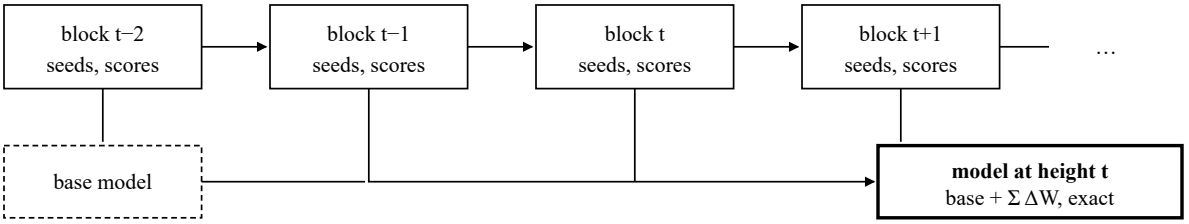


Figure 2: The model at any height is the pinned base plus every recorded update before it. Blocks carry about 2 KB each; the weights they define are never sent.

Exact agreement across devices needs exact noise. Vectors are drawn from the Philox counter-based generator [7] and shaped with the network's own logarithm, sine and cosine, built from operations IEEE 754 rounds correctly, so a CPU and any graphics card produce the same numbers from the same seed. A worker keeps its running change in 32-bit floating point; against an exact 64-bit rebuild of the same history, the change differed by at most 7.5×10^{-9}. Because a job may start from other jobs pinned by their final state, models can be chained or combined on chain, each still a function of the blocks.

6. Training Plan

Evolution makes most of its gain on a task early. On a test network, each of three tasks gained nearly all it would in its first ten to fifteen rounds and changed little over the next thirty: broad changes, such as answering in the required format, are found by many random directions, and narrow ones by few. A job is therefore short, twelve rounds by default, and the next job takes the next task, starting from the previous job's final change at full weight. Nineteen tasks run in turn and then repeat: Countdown puzzles and arithmetic, generated as needed, and sixteen public datasets covering science, commonsense, word-problem mathematics, trivia, reading, sentiment, news topics and emotion. A pass takes about seventy minutes on one graphics card.

An earlier design trained tasks side by side and started the next cycle from their mean. A mean of k jobs keeps only $1/k$ of each job's change, and on the test network the next cycle started below where the single jobs had reached. Chaining keeps every change at full strength; two datasets, GSM8K [8] and ARC-Challenge, are never trained on and measure whether gains carry over.

7. Incentive

Evo Hash runs on Vibe Hash's ledger, consensus and pay [2]. Validators take turns leading rounds, every other validator replays the round's record before voting, and a block is final once validators holding more than two thirds of the stake have signed it. Workers stake VIBE, are paid for units that pass checking in proportion to their work, and carry a reputation that rises with verified rounds and falls with missed ones. The requester pays only for rounds that accepted work; an Evo

job's fee is 1 VIBE a round, and a round in which no unit is accepted costs nothing. No role holds the model, prepares data for others, or decides the update: every node derives it from the block.

8. Calculations

Block size. An entry holds a 64-bit seed and, for each accepted pair, its index and coefficient, about 2 KB for 64 pairs as canonical JSON. The 357,826,560 weights it changes occupy about 716 MB in 16-bit floating point, so a generation is recorded about 350,000 times more compactly than its effect. A gradient step on the same model sends the full 716 MB from every participant.

Compute. A round generates up to $128 \times 16 \times 64 = 131{,}072$ tokens. At about 2 floating-point operations per weight per token, that is near 10^{14} operations, consistent with the eighteen seconds measured per generation on one RTX 4070 SUPER, rebuild included.

Rebuild. Applying a round on a CPU forms P outer products for each of 168 matrices, about $64 \times 357{,}826{,}560 \approx 2.3 \times 10^{10}$ multiply-adds, plus the round's noise. A worker that joins late rebuilds every round of the job's history, including its parents', which on the test network took about a quarter of an hour for 300 rounds; workers therefore save their rebuilt change to disk and resume from it.

Comparison. In a matched test on Countdown, ES with shared prompts and replay fitness reached about 24% in under two minutes, while GRPO [9], a gradient method, needed five minutes for 22.9%. GRPO pulled ahead in longer runs, reaching 33% at fifteen minutes, which is why Evo Hash's jobs are short.

9. Results

A test network on a single RTX 4070 SUPER trained Qwen2.5-0.5B-Instruct for 124 rounds over three tasks. Every one of 952 units passed checking. Scores in Table 1 are the share of all members' answers that were right, on sixteen new problems each round, so single rounds vary widely; fixed held-out sets will give the cleaner measure.

Task	First round	Last five rounds	Rounds
ARC-Easy	36.6%	63.3%	44
Countdown	9.5%	18.8%	44
Arithmetic	1.7%	9.1%	45

Table 1: Training reward in the test network's first cycle. Forty-three generations of one job occupied 95.8 KB of chain.

10. Limitations

Answer checks. Workers answer in 16-bit arithmetic on a fast engine, and checkers replay the answers through the same engine, which always computes 256 rows at a time, so a checker rounds exactly as the worker did: on one kind of graphics card, 256 honest

members over four tasks all had a largest gap of exactly 0, and the margin is 0.1. Two limits remain. In 16-bit arithmetic two tokens' scores can tie exactly, as 14 of 1,439 forged answers to multiple-choice questions did, and no margin detects a tie; and another kind of graphics card rounds differently and must be measured before it joins.

Selection. A worker that withholds units, or a checker whose rejections go unrefereed, can influence which pairs enter an update; simulation shows the influence is real. Refereed rejections and units given to more than one worker are needed before an open network.

Worker memory. A worker keeps each job's change in 32-bit floating point, 1.4 GB for this model, and the test network's workers used about 9 to 20 GB of memory.

Method. Gains flatten after the first rounds of a task, gradient methods win long single-task runs, and results so far are from one 0.5-billion-parameter model on short-answer tasks.

Status. Evo Hash runs on local test networks only; the public network does not yet accept Evo jobs. The software has not been audited, and VIBE has no cash value.

11. Conclusion

We have proposed a chain whose mining trains a model and whose blocks are that model. Training by evolution strategies reduces a step to seeds and scores, which fit in a block and which any node can turn into the same update, so the trained weights are rebuilt rather than stored or trusted. Members are defined exactly, from seeds no worker can choose; their answers are spot-checked by replay and regraded by code; and each generation's record is a few kilobytes against the hundreds of megabytes it changes. A test network implementing this design ships in Vibe Hash 0.5.12 as Evo Hash.

References

[1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008.

[2] C. Heideman, "Vibe Hash: Proof of Useful Work by Verifiable Neural Network Training," vibehash.org, 2026.

[3] T. Salimans, J. Ho, X. Chen, S. Sidor and I. Sutskever, "Evolution Strategies as a Scalable Alternative to Reinforcement Learning," arXiv:1703.03864, 2017.

[4] X. Qiu et al., "Evolution Strategies at Scale: LLM Fine-Tuning Beyond Reinforcement Learning," arXiv:2509.24372, 2025.

[5] "EGGROLL: Evolution Strategies at the Hyperscale," arXiv:2511.16652, 2025.

[6] P. Rajpurkar, J. Zhang, K. Lopyrev and P. Liang, "SQuAD: 100,000+ Questions for Machine Comprehension of Text," in Proc. EMNLP, 2016.

[7] J. K. Salmon, M. A. Moraes, R. O. Dror and D. E. Shaw, "Parallel Random Numbers: As Easy as 1, 2, 3," in Proc. SC11, 2011.

[8] K. Cobbe et al., "Training Verifiers to Solve Math Word Problems," arXiv:2110.14168, 2021.

[9] Z. Shao et al., "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models," arXiv:2402.03300, 2024.

[10] S. Malladi et al., "Fine-Tuning Language Models with Just Forward Passes," arXiv:2305.17333, 2023.

[11] Z. Qin et al., "Federated Full-Parameter Tuning of Billion-Sized Language Models with Communication Cost under 18 Kilobytes," in Proc. ICML, 2024.

Disclaimer: All product names, logos, and brands are property of their respective owners and are used purely for identification. Vibe Hash isn't affiliated with or endorsed by them.